



**Министерство науки и высшего образования Российской
Федерации**

**Федеральное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1

по дисциплине «>

Тема

Студент Прохоров С.Р.

Группа ИУ7-Х6Б

Преподаватель

Москва, 2026

Содержание

ВВЕДЕНИЕ	5
1 Аналитическая часть	6
1.1 Анализ предметной области	6
1.1.1 Сравнительный анализ существующих конкурентов	6
1.2 Формализация задачи	7
1.3 Формализация данных	8
1.4 Формализация ролей	10
1.5 Анализ моделей баз данных	11
1.5.1 Дореляционные модели	11
1.5.2 Реляционная модель данных	12
1.5.3 Постреляционные модели	12
2 Конструкторская часть	14
2.1 Описание сущностей базы данных	14
2.2 Ролевая модель	20
2.3 Используемые триггеры и процедура	20
3 Технологическая часть	24
3.1 Структура программы	24
3.2 Программная реализация	24
3.2.1 Создание базы данных	24
3.2.2 Создание ролей	33
3.2.3 Создание триггеров и хранимой процедуры	33
3.3 Интерфейс программной реализации	37

3.3.1	Интерфейс взаимодействия с базой данных	37
3.3.2	Пользовательский интерфейс	38
ЗАКЛЮЧЕНИЕ		41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		42
ПРИЛОЖЕНИЕ А		44

ВВЕДЕНИЕ

<цель и задачи; можно вводное слово к работе>

1 Аналитическая часть

1.1 Анализ предметной области

Организация и проведение спортивных турниров – это сложный многоуровневый процесс, требующий координации множества участников и ресурсов. Предметная область охватывает планирование соревнований, регистрацию участников, составление расписания матчей, назначение судей и фиксацию результатов.

Многие организаторы любительских и полупрофессиональных турниров используют для учета электронные таблицы или бумажные носители. Это приводит к ряду проблем:

- высокая вероятность человеческой ошибки при вводе данных и подсчете очков;
- сложность оперативного обновления турнирных таблиц и сеток;
- отсутствие единого информационного пространства для игроков, судей и болельщиков;
- трудности в оповещении участников об изменениях в расписании.

Внедрение автоматизированной системы управления турнирами позволит централизованно хранить информацию, автоматически создавать турнирные сетки, рассчитывать статистику и обеспечивать мгновенный доступ к результатам.

1.1.1 Сравнительный анализ существующих конкурентов

Для анализа выбраны три известные российские платформы: Наградион [1], Goalstream [2] и JoinSport [3].

Анализ конкурентов показывает, что существующие решения либо слишком сложны и ориентированы на официальные федерации, либо имеют узкую специализацию и перегружены социальными функциями. Проектируемая база данных и приложение должны обладать основными функциями рассмотренных конкурентов — занесением заявок на участие в турниры и результатов матчей, турниров, а также четким разделением ролей, — при этом оставаясь простыми в освоении для организаторов небольших и средних турниров, т. е. не требуя специализированного обучения по пользованию программным обеспечением.

Таблица 1.1 — Сравнительный анализ систем управления турнирами

Критерий	Наградион [1]	Goalstream [2]	JoinSport [3]
Целевая аудитория	Региональные федерации, крупные лиги.	Любительские командные виды спорта (чаще футбол).	Организаторы турниров, спортивные школы.
Поддержка форматов	Максимально гибкая, для множества видов спорта.	Круговая система, плей-офф, группы.	Групповые этапы, плей-офф, шахматная система.
Пользовательские роли	Администратор, судья, статистик, игрок.	Игрок, менеджер, организатор, болельщик.	Администратор, тренер, игрок, судья.
Интерфейс	Перегруженный интерфейс из-за обилия настроек.	Удобный интерфейс с акцентом на социальные сети.	Современный интерфейс, конструктор сайтов.
Интеграция данных	Выгрузка официальных протоколов, статистика.	Базовый экспорт, есть мобильное приложение.	Хороший API, виджеты для сторонних сайтов.
Недостатки	Высокий порог вхождения, сложность для малых турниров.	Упор на функционал соцсети, узкая специализация.	Платный доступ к модулям, избыточность для разовых турниров.

1.2 Формализация задачи

Необходимо спроектировать базу данных для хранения информации о пользователях, турнирах, командах, игроках, матчах, спортивных площадках и статистике. Требуется разработать приложение, предоставляющее интерфейс для просмотра, добавления, изменения и удаления информации, хранящейся в базе данных. Необходимо реализовать три вида ролей – гость, представитель команды и организатор турниров.

1.3 Формализация данных

Основываясь на анализе предметной области, можно выделить следующие категории данных:

- пользователь;
- турнир;
- организатор турнира;
- команда;
- игрок;
- судья;
- тренер;
- матч;
- спортивная площадка.

Сведения о каждой категории данных содержится в таблице 1.2.

Таблица 1.2 — Категории и сведения о данных

Категория	Сведения
Организатор	Название компании, электронная почта, номер телефона
Спортивная площадка	Название локации, город, вместимость
Команда	Название команды, страна, дата основания
Игрок	Привязка к команде, никнейм, настоящее имя, дата рождения
Тренер	Привязка к команде, имя, фамилия, опыт работы (в годах)
Судья	ФИО, категория, опыт работы (в годах)
Турнир	Организатор, название, вид спорта, дата начала, дата окончания, призовой фонд, статус
Заявка на турнир	Турнир, команда, дата регистрации, статус заявки (ожидает, принята, отклонена)
Матч	Турнир, локация, дата и время проведения, стадия турнира, статус матча, команды-участницы и их счет, назначенные судьи и их роли

На рисунке 1.1 приведена ER-диаграмма сущностей в нотации Чена.

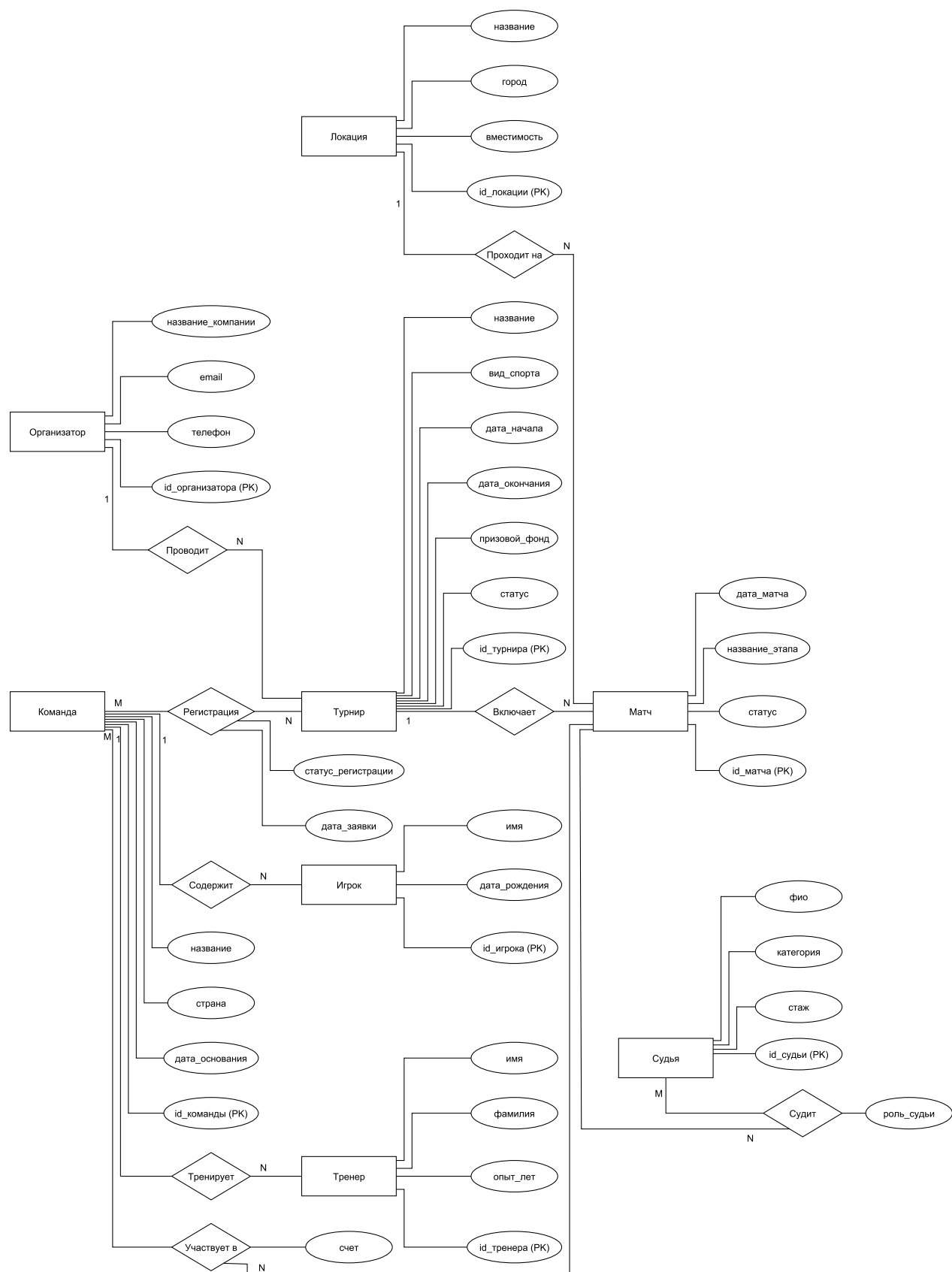


Рисунок 1.1 — ER-диаграмма в нотации Чена

1.4 Формализация ролей

В системе выделяются 3 типа пользователей:

- гость – неавторизованный пользователь, обладающий возможностями просматривать список турниров, турнирные таблицы, расписание матчей, составы команд и статистику игроков;
- участник (представитель команды) – авторизованный пользователь, обладающий возможностью просматривать общую информацию, подавать и редактировать заявку своей команды на участие в турнире, управлять списком игроков команды;
- организатор (администратор) – авторизованный пользователь, обладающий возможностью просматривать, добавлять и менять данные турниров, команд, игроков, площадок, формировать расписание матчей, вносить результаты игр и управлять ролями пользователей.

На рисунке 1.2 приведена Use-case диаграмма.

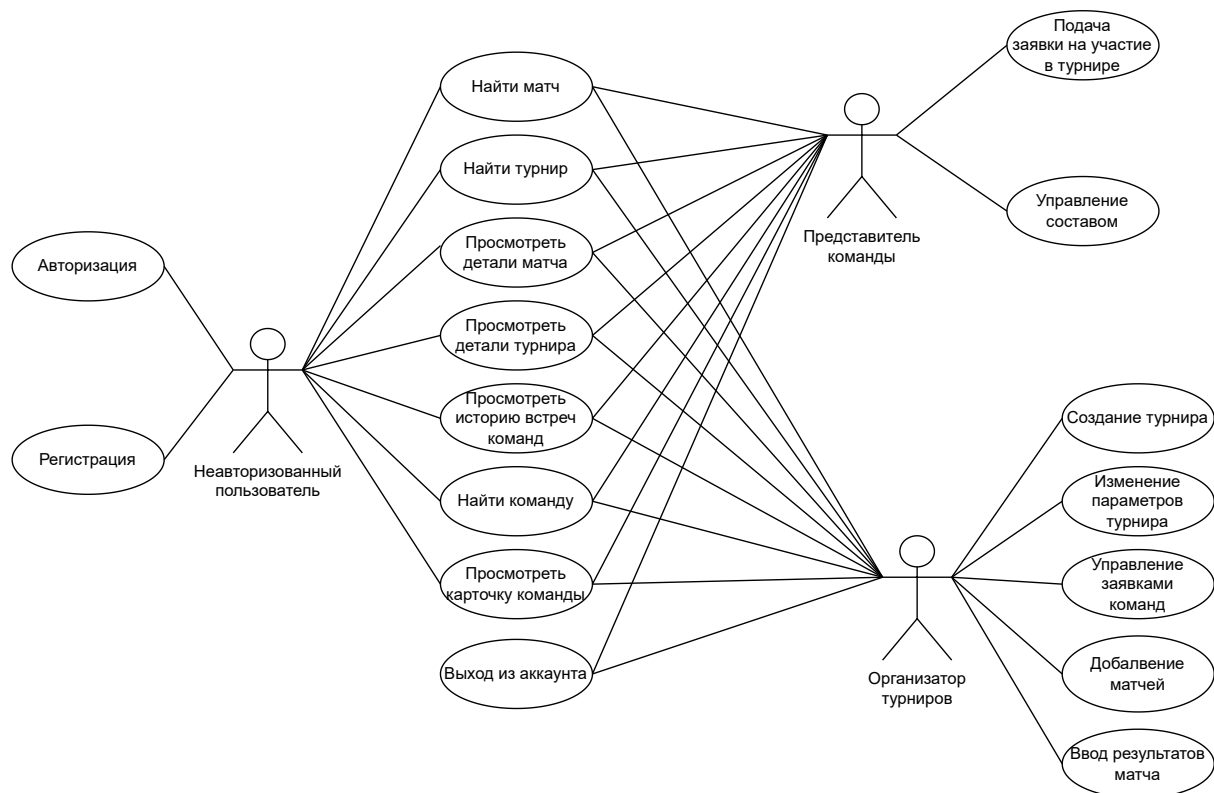


Рисунок 1.2 — Use-case диаграмма

1.5 Анализ моделей баз данных

Основой любой информационной системы является база данных. Ключевое отличие этих моделей заключается в способе установления связей между данными – современные архитектуры информационных систем, выделяя до 13 и более различных категорий баз данных [5].

1.5.1 Дореляционные модели

Иерархические базы данных имеют форму деревьев, состоящих из дуг – связей, и узлов – элементов данных [4]. В такой структуре объекты находятся в строгих отношениях «предок – потомок». Главной особенностью и ограничением иерархической модели является то, что объект-потомок обязательно имеет только одного предка, в то время как предок может иметь несколько потомков.

Сетевая модель допускает более сложные связи между множествами. Основное различие этих моделей состоит в том, что в сетевой модели запись может быть членом более чем одного группового отношения. Согласно этой модели каждое групповое отношение именуется, тип группового отношения задается

его именем и определяет свойства общие для всех экземпляров данного типа. Экземпляр группового отношения представляется записью-владельцем и множеством подчиненных записей. При этом экземпляр записи не может быть членом двух экземпляров групповых отношений одного типа.

1.5.2 Реляционная модель данных

Основоположником реляционной модели является Э. Кодд. Основным достоинством реляционного подхода является его строгая математическая база, основанная на теории множеств и логике предикатов [6].

Реляционная модель состоит из трех частей.

- 1) Структурная часть – фиксирует, что единственной структурой данных в БД является n -арное нормализованное отношение – таблица. Определяются понятия домена, атрибута, кортежа и первичного ключа.
- 2) Целостная часть – определяет два требования: требование целостности сущностей – каждый кортеж обязан обладать уникальным первичным ключом — и требование ссылочной целостности, которое означает, что внешний ключ одной таблицы должен строго соответствовать существующему первичному ключу другой таблицы [6, 7].
- 3) Манипуляционная часть – утверждает механизмы манипулирования данными на основе реляционной алгебры и реляционного исчисления.

1.5.3 Постреляционные модели

Постреляционные модели снимают ограничение неделимости данных, допуская использование многозначных полей. Набор значений таких полей фактически представляет собой самостоятельную таблицу, встроенную в основную [7], что обеспечивает высокую наглядность представления сложных объектов.

На данный момент активно применяются документо-ориентированные, графовые и ключ-значение базы данных [5]. Они обеспечивают гибкость схем данных и высокую масштабируемость, однако зачастую не имеют строгую транзакционную целостность и поддержку сложных перекрестных запросов.

Вывод

В ходе анализа предметной области организации спортивных турниров были выделены основные сущности: организатор, спортивная площадка, команда, игрок, тренер, судья, турнир, заявка на турнир и матч. Была поставлена задача, а также проведена формализация ролей. Были выделены 3 основные роли – неавторизованный пользователь, представитель команды и организатор турниров, для каждой из трёх ролей были определены их возможные действия при работе с базой данных.

Постреляционные решения не подходят для учетных задач. При этом строгие требования реляционной модели к ограничению целостности по ссылкам [6, 7] подходят для разрабатываемой системы.

Реляционная модель, основанная на реляционной алгебре, гарантирует надёжность сложных вычислений. В связи с этим в качестве основы для проектирования разрабатываемой системы выбрана реляционная модель базы данных.

2 Конструкторская часть

2.1 Описание сущностей базы данных

В соответствии с таблицей 1.2, содержащей данные, которые должны находиться в базе данных, можно выделить следующие таблицы:

- 1) organizer – таблица организаторов турниров;
- 2) location – таблица площадок (локаций) проведения матчей;
- 3) team – таблица спортивных команд;
- 4) referee – таблица судей;
- 5) tournament – таблица спортивных турниров;
- 6) match – таблица матчей турнира;
- 7) player – таблица игроков команд;
- 8) coach – таблица тренеров команд;
- 9) tournament_registration – таблица регистраций команд на турнир;
- 10) match_participant – таблица команд-участников матчей и их результатов;
- 11) match_referee – таблица назначения судей на конкретные матчи.

На основе информации о выбранной СУБД и представленной на рисунке 1.1 диаграмме сущность-связь, мы определим для каждой из указанных таблиц структуру столбцов, их типы и установим соответствующие ограничения, которые представлены в таблицах 2.1–2.9.

Таблица 2.1 — Информация о столбцах таблицы организаторов (organizer)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор организатора
company_name	text	NOT NULL	Название компании
email	text	NOT NULL, UNIQUE, CHECK (формат email)	Электронная почта
phone	text	CHECK(формат телефона)	Номер телефона

Таблица 2.2 — Информация о столбцах таблицы площадок (location)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор площадки
name	text	NOT NULL	Название
city	text	NOT NULL	Город
capacity	bigint	CHECK (capacity > 0)	Вместимость

Таблица 2.3 — Информация о столбцах таблицы команд (team)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор команды
name	text	NOT NULL	Название команды
country	text	NOT NULL	Страна
foundation_date	date	NOT NULL	Дата основания

Таблица 2.4 — Информация о столбцах таблицы судей (referee)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор судьи
full_name	text	NOT NULL	ФИО судьи
category	text	NOT NULL	Категория
experience_years	int	DEFAULT 0, CHECK (≥ 0)	Опыт работы (в годах)

Таблица 2.5 — Информация о столбцах таблицы турниров (tournament)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор турнира
organizer_id	bigint	NOT NULL, FK (organizer) ON DELETE CASCADE	Идентификатор организатора
name	text	NOT NULL	Название турнира
sport_type	text	NOT NULL	Вид спорта
start_date	date	NOT NULL, CHECK ($\leq$ end_date)	Дата начала
end_date	date	NOT NULL	Дата окончания
prize_pool	numeric	DEFAULT 0.00, CHECK ($\geq$ 0)	Призовой фонд
status	text	DEFAULT 'registration', CHECK (status IN ...)	Статус турнира

Таблица 2.6 — Информация о столбцах таблицы матчей (match)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор матча
tournament_id	bigint	NOT NULL, FK (tournament) ON DELETE CASCADE	Идентификатор турнира
location_id	bigint	FK (location) ON DELETE SET NULL	Идентификатор площадки
match_date	timestamp	NOT NULL	Дата и время проведения
stage_name	text	NOT NULL	Стадия турнира
status	text	DEFAULT 'scheduled', CHECK (status IN ...)	Статус матча

Таблица 2.7 — Информация о столбцах таблицы игроков (player)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор игрока
team_id	bigint	FK (team) ON DELETE SET NULL	Идентификатор команды
nickname	text	NOT NULL	Никнейм
real_name	text	NOT NULL	Настоящее имя
birth_date	date	NOT NULL	Дата рождения

Таблица 2.8 — Информация о столбцах таблицы тренеров (coach)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор тренера
team_id	bigint	FK (team) ON DELETE SET NULL	Идентификатор команды
first_name	text	NOT NULL	Имя
last_name	text	NOT NULL	Фамилия
experience_years	int	DEFAULT 0, CHECK (≥ 0)	Опыт работы (в годах)

Таблица 2.9 — Информация о столбцах таблицы судей матча (match_referee)

Столбец	Тип данных	Ограничения	Значение
match_id	bigint	NOT NULL, PK, FK (match) ON DELETE CASCADE	Идентификатор матча
referee_id	bigint	NOT NULL, PK, FK (referee) ON DELETE CASCADE	Идентификатор судьи
role	text	NOT NULL	Роль судьи

Таблица 2.10 — Информация о столбцах таблицы регистрации на турнир (tournament_registration)

Столбец	Тип данных	Ограничения	Значение
tournament_id	bigint	NOT NULL, PK, FK (tournament) ON DELETE CASCADE	Идентификатор турнира
team_id	bigint	NOT NULL, PK, FK (team) ON DELETE CASCADE	Идентификатор команды
registration_date	timestamp	DEFAULT current_timestamp	Дата регистрации
status_registration	text	DEFAULT 'pending', CHECK (status IN ...)	Статус заявки

Таблица 2.11 — Информация о столбцах таблицы участников матча (match_participant)

Столбец	Тип данных	Ограничения	Значение
match_id	bigint	NOT NULL, PK, FK (match) ON DELETE CASCADE	Идентификатор матча
team_id	bigint	NOT NULL, PK, FK (team) ON DELETE CASCADE	Идентификатор команды
score	int	DEFAULT 0, CHECK (≥ 0)	Счет команды

На рисунке 2.1 приведена ER-диаграмма базы данных.



Рисунок 2.1 — Схема базы данных

2.2 Ролевая модель

Для реализации описанной логики на уровне базы данных (PostgreSQL) проектируется ролевая модель, включающая 3 роли базы данных. Разграничение доступа к объектам (таблицам и представлениям) выглядит следующим образом.

- 1) Неавторизованный пользователь – имеет права доступа только на чтение к таблицам tournament, match, team, player, location, match_participant. Запрещены любые операции изменения данных.
- 2) Представитель команды – те же права, что и у неавторизованного пользователя, при этом появляются возможности для добавления, обновления и удаления данных из таблиц tournament_registration player
- 3) Организатор турнира – имеет полный доступ в базе данных.

2.3 Используемые триггеры и процедура

В базе данных присутствуют два триггера и одна хранимая процедура. Один триггер активируется автоматически после завершения матча-финала турнира и автоматически завершает турнир, чей финал был окончен, что позволяет обеспечить актуальность данных. Второй триггер позволяет организаторам автоматически создавать расписание группового этапа. Хранимая процедура автоматизирует процесс регистрации команд на турнир.

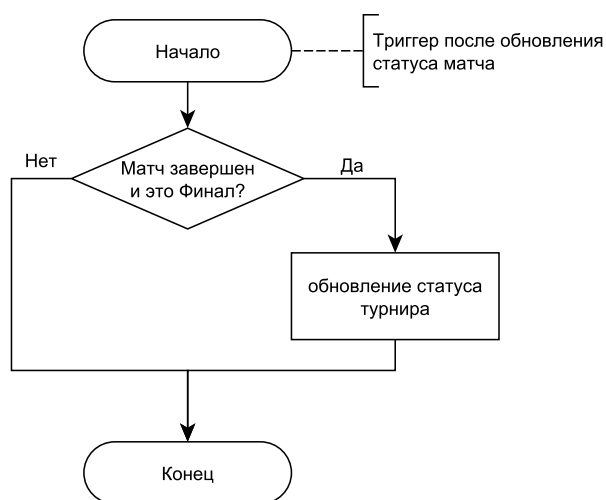


Рисунок 2.2 — Схема алгоритма триггера, обновляющего статус турнира после конца матча-финала

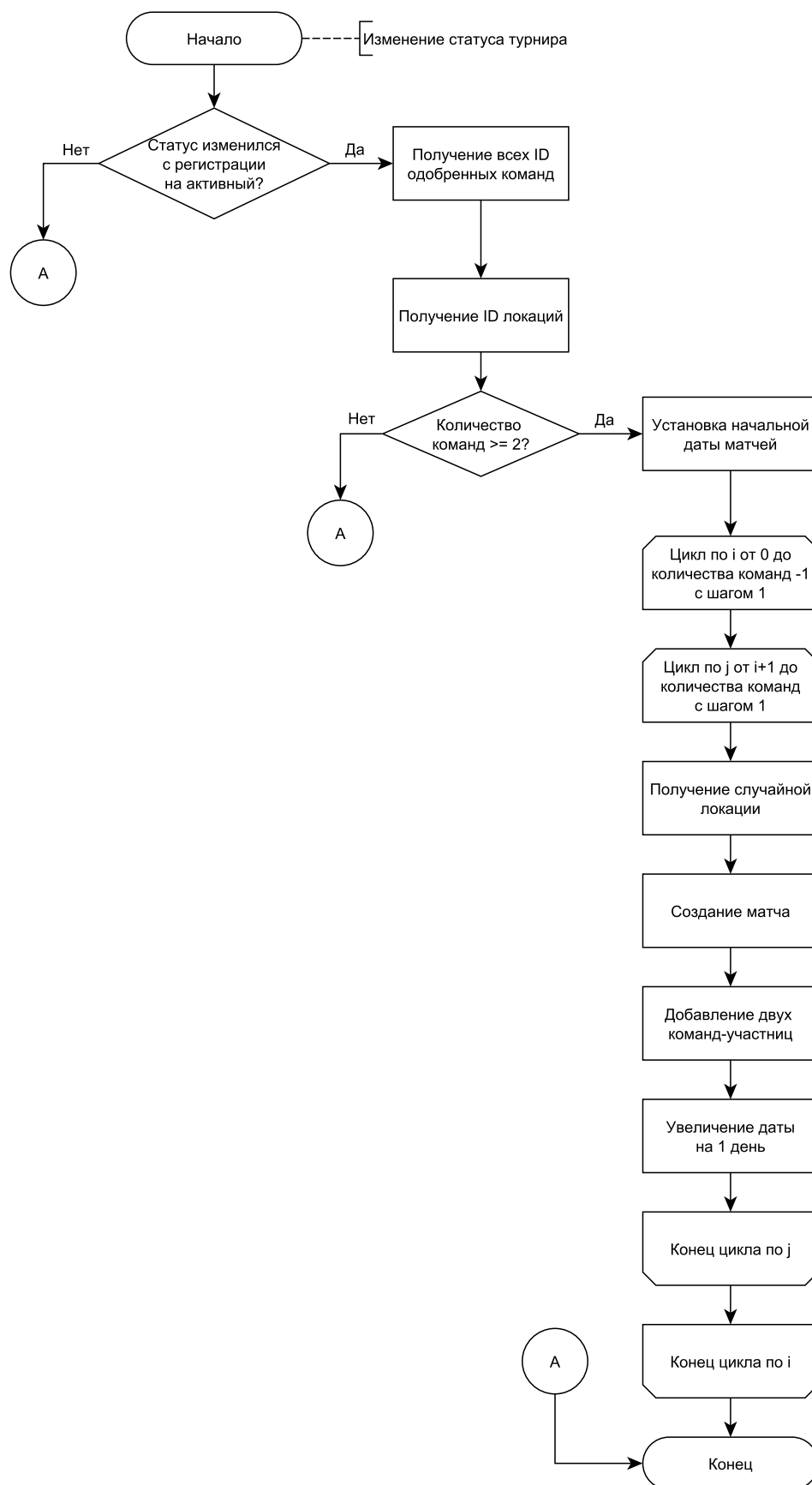


Рисунок 2.3 — Схема алгоритма триггера, создающего расписание группового этапа матчей

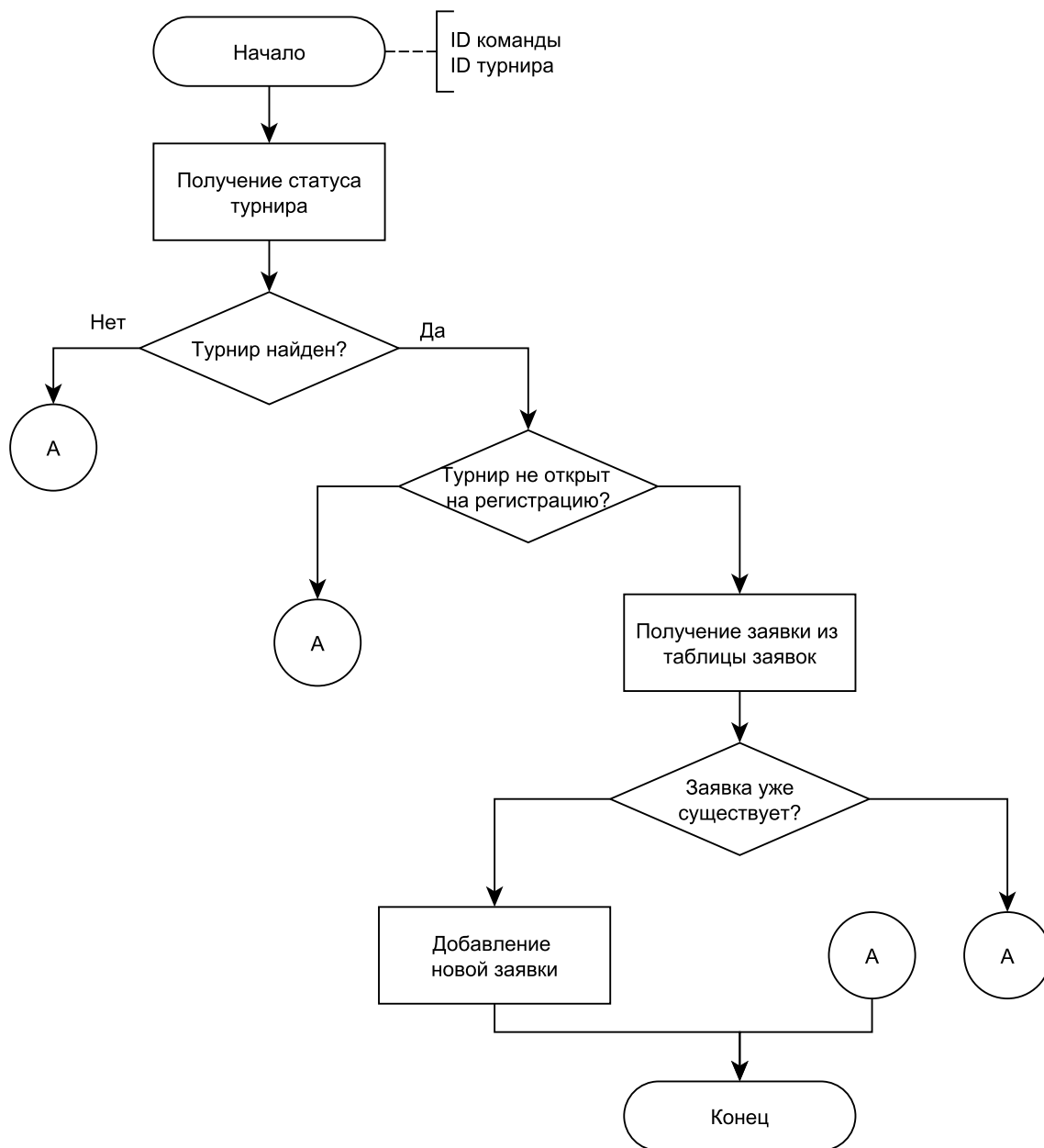


Рисунок 2.4 — Схема алгоритма хранимой процедуры, которая осуществляет регистрацию команды на турнир

Вывод

В данном разделе были описаны все сущности базы данных на основе анализа предметной области, представлена ER-диаграмма сущностей, описана ролевая модель. Также был спроектирован необходимый триггер.

3 Технологическая часть

Для программной реализации серверной части приложения, которая осуществляет запросы к базе данных был выбран язык TypeScript [8] с библиотекой Express [9], предназначенной для создания веб-серверов и обработки HTTP-запросов. Для реализации пользовательской части использовался язык разметки HTML [10], язык таблиц стилей CSS [11] и для реализации взаимодействия пользователя с интерфейсом использовался язык Typescript [8]. Для сборки и запуска пользовательской части приложения использовался инструмент Vite [12]. В качестве системы управления базами данных использовалась PostgreSQL [13], обеспечивающая хранение и обработку данных приложения. Для реализации механизма кеширования и хранения временных данных использовалась система Redis [14]. Разработка осуществлялась в программе VScode [15].

3.1 Структура программы

На рисунке 3.1 представлена диаграмма классов разработанной программы.

Рисунок 3.1 — Диаграмма разработанной программы

3.2 Программная реализация

3.2.1 Создание базы данных

На листинге 3.1 представлен код программы для создания таблицы пользователей.

Листинг 3.1 — Создание таблицы пользователей

```
1      create table if not exists sport.users (  
2          id bigserial,  
3          email text,  
4          password_hash text,  
5          role sport.user_role,  
6          created_at timestamp default current_timestamp  
7      );
```

На листинге 3.2 представлен код программы для создания таблицы менеджеров.

Листинг 3.2 — Создание таблицы менеджеров

```
1      create table if not exists sport.manager (  
2          id bigserial primary key,  
3          user_id bigint unique,  
4          team_id bigint  
5      );
```

На листинге 3.3 представлен код программы для создания таблицы организаторов.

Листинг 3.3 — Создание таблицы организаторов

```
1      create table if not exists sport.organizer (  
2          id bigserial,  
3          user_id bigint unique,  
4          company_name text,  
5          email text,  
6          phone text  
7      );
```

На листинге 3.4 представлен код программы для создания таблицы локаций.

Листинг 3.4 — Создание таблицы локаций

```
1      create table if not exists sport.location (  
2          id bigserial,  
3          name text,  
4          city text,  
5          capacity bigint  
6      );
```

На листинге 3.5 представлен код программы для создания таблицы команд.

Листинг 3.5 — Создание таблицы команд

```
1      create table if not exists sport.team (  
2          id bigserial,  
3          name text,  
4          country text,
```



```
5      foundation_date date
6  );
```

На листинге 3.6 представлен код программы для создания таблицы судей.

Листинг 3.6 — Создание таблицы судей

```
1      create table if not exists sport.referee (
2      id bigserial,
3      full_name text,
4      category text,
5      experience_years int
6  );
```

На листинге 3.7 представлен код программы для создания таблицы турниров.

Листинг 3.7 — Создание таблицы турниров

```
1      create table if not exists sport.tournament (
2      id bigserial,
3      organizer_id bigint,
4      name text,
5      sport_type text,
6      start_date date,
7      end_date date,
8      prize_pool numeric,
9      status text
10     );
```

На листинге 3.8 представлен код программы для создания таблицы матчей.

Листинг 3.8 — Создание таблицы матчей

```
1      create table if not exists sport.match (
2      id bigserial,
3      tournament_id bigint,
4      location_id bigint,
5      match_date timestamp,
6      stage_name text,
7      status text
8  );
```

На листинге 3.9 представлен код программы для создания таблицы игроков.

Листинг 3.9 — Создание таблицы игроков

```
1      create table if not exists sport.player (  
2          id bigserial,  
3          team_id bigint,  
4          real_name text,  
5          birth_date date  
6      );
```

На листинге 3.10 представлен код программы для создания таблицы тренеров.

Листинг 3.10 — Создание таблицы тренеров

```
1      create table if not exists sport.coach (  
2          id bigserial,  
3          team_id bigint,  
4          first_name text,  
5          last_name text,  
6          experience_years int  
7      );
```

На листинге 3.11 представлен код программы для создания таблицы регистрации на турнир.

Листинг 3.11 — Создание таблицы регистрации на турнир

```
1      create table if not exists sport.tournament_registration (  
2          tournament_id bigint,  
3          team_id bigint,  
4          registration_date timestamp,  
5          status_registration text  
6      );
```

На листинге 3.12 представлен код программы для создания таблицы участников матчей.

Листинг 3.12 — Создание таблицы участников матчей

```
1      create table if not exists sport.match_participant (  
2          match_id bigint,  
3          team_id bigint,  
4          score int  
5      );
```

На листинге 3.13 представлен код программы для создания таблицы судей матчей.

Листинг 3.13 — Создание таблицы судей матчей

```
1      create table if not exists sport.match_referee (  
2      match_id bigint,  
3      referee_id bigint,  
4      role text  
5      );
```

На листинге 3.14 представлен код программы для наложения ограничений на таблицу пользователей.

Листинг 3.14 — Создание ограничений для таблицы пользователей

```
1      alter table sport.users  
2      add constraint pk_user_id primary key (id),  
3      alter column email set not null,  
4      alter column password_hash set not null,  
5      alter column role set not null,  
6      add constraint uq_users_email unique (email),  
7      add constraint valid_email check (email ~ '^[a-zA-Z0-9._]+@[a-zA-Z  
    ]+\.[a-zA-Z0-9]{2,}$');
```

На листинге 3.15 представлен код программы для наложения ограничений на таблицу организаторов.

Листинг 3.15 — Создание ограничений для таблицы организаторов

```
1      alter table sport.organizer  
2      add constraint pk_organizer_id primary key (id),  
3      add constraint uq_organizer_email unique (email),  
4      add constraint valid_email check (email ~ '^[a-zA-Z0-9._]+@[a-zA-Z  
    ]+\.[a-zA-Z0-9]{2,}$');  
  
5  
6      alter table sport.organizer  
7      alter column id set not null,  
8      alter column company_name set not null,  
9      alter column email set not null;
```

На листинге 3.16 представлен код программы для наложения ограничений на таблицу локаций.

Листинг 3.16 — Создание ограничений для таблицы локаций

```
1      alter table sport.location
2      add constraint pk_location_id primary key (id),
3      add constraint chk_location_capacity check (capacity > 0);
4
5      alter table sport.location
6      alter column id set not null,
7      alter column name set not null,
8      alter column city set not null;
```

На листинге 3.17 представлен код программы для наложения ограничений на таблицу команд.

Листинг 3.17 — Создание ограничений для таблицы команд

```
1      alter table sport.team
2      add constraint pk_team_id primary key (id);
3
4      alter table sport.team
5      alter column id set not null,
6      alter column name set not null,
7      alter column country set not null;
```

На листинге 3.18 представлен код программы для наложения ограничений на таблицу судей.

Листинг 3.18 — Создание ограничений для таблицы судей

```
1      alter table sport.referee
2      add constraint pk_referee_id primary key (id),
3      add constraint chk_referee_experience check (experience_years >= 0);
4
5      alter table sport.referee
6      alter column id set not null,
7      alter column full_name set not null,
8      alter column category set not null,
9      alter column experience_years set default 0;
```

На листинге 3.19 представлен код программы для наложения ограничений на таблицу турниров.

Листинг 3.19 — Создание ограничений для таблицы турниров

```

1      alter table sport.tournament
2      add constraint pk_tournament_id primary key (id),
3      add constraint fk_tournament_organizer foreign key (organizer_id)
references sport.organizer (id) on delete cascade,
4      add constraint chk_tournament_dates check (start_date <= end_date),
5      add constraint chk_tournament_prize check (prize_pool >= 0),
6      add constraint chk_tournament_status check (status in ('registration',
'active', 'completed', 'cancelled'));
7
8      alter table sport.tournament
9      alter column id set not null,
10     alter column organizer_id set not null,
11     alter column name set not null,
12     alter column sport_type set not null,
13     alter column start_date set not null,
14     alter column end_date set not null,
15     alter column prize_pool set default 0.00,
16     alter column status set default 'registration';

```

На листинге 3.20 представлен код программы для наложения ограничений на таблицу матчей.

Листинг 3.20 — Создание ограничений для таблицы матчей

```

1      alter table sport.match
2      add constraint pk_match_id primary key (id),
3      add constraint fk_match_tournament foreign key (tournament_id)
references sport.tournament (id) on delete cascade,
4      add constraint fk_match_location foreign key (location_id) references
sport.location (id) on delete set null,
5      add constraint chk_match_status check (status in ('scheduled', '
ongoing', 'finished', 'cancelled'));
6
7      alter table sport.match
8      alter column id set not null,
9      alter column tournament_id set not null,
10     alter column match_date set not null,
11     alter column stage_name set not null,
12     alter column status set default 'scheduled';

```

На листинге 3.21 представлен код программы для наложения ограничений

на таблицу игроков.

Листинг 3.21 — Создание ограничений для таблицы игроков

```
1      alter table sport.player
2      add constraint pk_player_id primary key (id),
3      add constraint fk_player_team foreign key (team_id) references sport.
team (id) on delete set null;
4
5      alter table sport.player
6      alter column id set not null,
7      alter column real_name set not null,
8      alter column birth_date set not null;
```

На листинге 3.22 представлен код программы для наложения ограничений на таблицу тренеров.

Листинг 3.22 — Создание ограничений для таблицы тренеров

```
1      alter table sport.coach
2      add constraint pk_coach_id primary key (id),
3      add constraint fk_coach_team foreign key (team_id) references sport.
team (id) on delete set null,
4      add constraint chk_coach_experience check (experience_years >= 0);
5
6      alter table sport.coach
7      alter column id set not null,
8      alter column first_name set not null,
9      alter column last_name set not null,
10     alter column experience_years set default 0;
```

На листинге 3.23 представлен код программы для наложения ограничений на таблицу регистрации на турнир.

Листинг 3.23 — Создание ограничений для таблицы регистрации на турнир

```
1      alter table sport.tournament_registration
2      add constraint pk_tournament_registration primary key (tournament_id,
team_id),
3      add constraint fk_reg_tournament foreign key (tournament_id)
references sport.tournament (id) on delete cascade,
4      add constraint fk_reg_team foreign key (team_id) references sport.team
(id) on delete cascade,
```

```

5      add constraint chk_reg_status check (status_registration in ('pending'
, 'approved', 'rejected'));
6
7      alter table sport.tournament_registration
8      alter column tournament_id set not null,
9      alter column team_id set not null,
10     alter column registration_date set default current_timestamp,
11     alter column status_registration set default 'pending';

```

На листинге 3.24 представлен код программы для наложения ограничений на таблицу участников матчей.

Листинг 3.24 — Создание ограничений для таблицы участников матчей

```

1      alter table sport.match_participant
2      add constraint pk_match_participant primary key (match_id, team_id),
3      add constraint fk_part_match foreign key (match_id) references sport.
match (id) on delete cascade,
4      add constraint fk_part_team foreign key (team_id) references sport.
team (id) on delete cascade,
5      add constraint chk_part_score check (score >= 0);
6
7      alter table sport.match_participant
8      alter column match_id set not null,
9      alter column team_id set not null,
10     alter column score set default 0;

```

На листинге 3.25 представлен код программы для наложения ограничений на таблицу судей матчей.

Листинг 3.25 — Создание ограничений для таблицы судей матчей

```

1      alter table sport.match_referee
2      add constraint pk_match_referee primary key (match_id, referee_id),
3      add constraint fk_ref_match foreign key (match_id) references sport.
match (id) on delete cascade,
4      add constraint fk_ref_referee foreign key (referee_id) references
sport.referee (id) on delete cascade;
5
6      alter table sport.match_referee
7      alter column match_id set not null,
8      alter column referee_id set not null,
9      alter column role set not null;

```

3.2.2 Создание ролей

На листинге 3.26 представлен код программы для создания системных ролей и назначения им прав доступа.

Листинг 3.26 — Создание ролей и распределение прав доступа

```
1      create role guest_role;
2      create role team_manager_role;
3      create role organizer_role;
4
5      grant usage on schema sport to guest_role, team_manager_role,
6      organizer_role;
7
8      grant select on all tables in schema sport to guest_role;
9
10     grant select on all tables in schema sport to team_manager_role;
11
12     grant insert, update, delete on sport.player to team_manager_role;
13     grant insert, update, delete on sport.tournament_registration to
14     team_manager_role;
15
16     grant all privileges on all tables in schema sport to organizer_role;
17
18     grant execute on procedure sport.start_tournament(bigint) to
19     organizer_role;
```

3.2.3 Создание триггеров и хранимой процедуры

На листинге 3.27 представлен код программы для создания хранимой процедуры регистрации команды на турнир.

Листинг 3.27 — Создание процедуры регистрации команды

```
1      create or replace procedure sport.register_team(p_team_id bigint,
2      p_tournament_id bigint)
3      language plpgsql
4      as $$
5      declare
6      v_status text;
```



```

6      v_exists boolean;
7      begin
8      select status into v_status
9      from sport.tournament
10     where id = p_tournament_id;
11
12     if not found then
13     raise exception 'Турнир % не найден', p_tournament_id;
14     end if;
15
16     if v_status != 'registration' then
17     raise exception 'Регистрация на турнир закрыта (текущий статус: %)',
v_status;
18     end if;
19
20     select exists(
21     select 1 from sport.tournament_registration
22     where tournament_id = p_tournament_id and team_id = p_team_id
23     ) into v_exists;
24
25     if v_exists then
26     raise exception 'Команда % уже подала заявку на турнир %', p_team_id,
p_tournament_id;
27     end if;
28
29     insert into sport.tournament_registration(tournament_id, team_id,
registration_date, status_registration)
30     values (p_tournament_id, p_team_id, current_timestamp, 'pending');
31     end;
32     $$;

```

На листинге 3.28 представлен код программы для создания триггера и триггерной функции автоматической генерации расписания турнира.

Листинг 3.28 — Создание триггера генерации расписания турнира

```

1      create or replace function sport.trg_generate_tournament_schedule()
2      returns trigger
3      language plpgsql
4      as $$
5      declare

```

```

6      v_team_ids bigint[];
7      v_loc_ids bigint[];
8      v_team_count int;
9      v_loc_count int;
10     v_match_id bigint;
11     v_loc_idx int;
12     v_date timestamp;
13     i int;
14     j int;
15     begin
16     if old.status = 'registration' and new.status = 'active' then
17
18     select array_agg(team_id) into v_team_ids
19     from sport.tournament_registration
20     where tournament_id = new.id and status_registration = 'approved';
21
22     select array_agg(id) into v_loc_ids
23     from sport.location;
24
25     v_team_count := coalesce(array_length(v_team_ids, 1), 0);
26     v_loc_count := coalesce(array_length(v_loc_ids, 1), 0);
27
28     if v_team_count >= 2 then
29     v_date := coalesce(new.start_date::timestamp, current_timestamp);
30
31     for i in 1 .. (v_team_count - 1) loop
32     for j in (i + 1) .. v_team_count loop
33
34     v_loc_idx := floor(random() * v_loc_count + 1)::int;
35
36     insert into sport.match (tournament_id, location_id, match_date,
stage_name, status)
37     values (
38     new.id,
39     v_loc_ids[v_loc_idx],
40     v_date,
41     'Групповой этап',
42     'scheduled'
43     ) returning id into v_match_id;
44

```

```

45     insert into sport.match_participant (match_id, team_id, score)
46     values (v_match_id, v_team_ids[i], null);
47
48     insert into sport.match_participant (match_id, team_id, score)
49     values (v_match_id, v_team_ids[j], null);
50
51     v_date := v_date + interval '1 day';
52
53     end loop;
54 end loop;
55 end if;
56 end if;
57
58     return new;
59 end;
60 $$;
61
62     drop trigger if exists trg_generate_schedule on sport.tournament;
63
64     create trigger trg_generate_schedule
65     after update of status on sport.tournament
66     for each row
67     execute function sport.trg_generate_tournament_schedule();

```

На листинге 3.29 представлен код программы для создания триггера и триггерной функции автоматического завершения турнира.

Листинг 3.29 — Создание триггера завершения турнира

```

1     create or replace function sport.trg_finish_tournament()
2     returns trigger
3     language plpgsql
4     as $$
5     begin
6         if new.status = 'finished' and old.status is distinct from 'finished'
and new.stage_name = 'Финал' then
7
8         update sport.tournament
9         set status = 'completed'
10        where id = new.tournament_id;
11

```

```

12      end if;
13
14      return new;
15      end;
16      $$;
17
18      drop trigger if exists trg_match_finish_tournament on sport.match;
19
20      create trigger trg_match_finish_tournament
21      after update of status on sport.match
22      for each row
23      execute function sport.trg_finish_tournament();

```

3.3 Интерфейс программной реализации

3.3.1 Интерфейс взаимодействия с базой данных

Для работы с базой данных был создан API при помощи библиотеки Express [9]. В программном интерфейсе реализованы методы для выполнения операций создания, чтения и удаления всех созданных сущностей в базе данных.

Методы взаимодействия с таблицей турниров базы данных:

- получение всех турниров;
- получение турнира по идентификатору;
- получение всех турниров по идентификатору организатора;
- создание турнира;
- обновление информации о турнире.

Методы взаимодействия с таблицей матчей базы данных:

- получение всех матчей с возможностью фильтрации;
- получение матча по идентификатору;
- получение всех матчей по идентификатору турнира;
- получение истории матчей между двумя заданными командами;
- получение списка команд с одобренными заявками для участия в турнире;
- создание матча;
- обновление информации о матче.

Методы взаимодействия с таблицей заявок на турниры базы данных:

- получение всех турниров, доступных для регистрации конкретной команды;
- получение всех заявок по идентификатору команды;
- получение всех заявок по идентификатору организатора;
- создание заявки на участие в турнире;
- обновление статуса заявки – подтверждение или отклонение.

Методы взаимодействия с таблицей команд базы данных:

- получение полной информации о команде по идентификатору;
- получение идентификатора команды по названию;
- получение названия команды по идентификатору;
- получение команды по идентификатору пользователя-менеджера.

Методы взаимодействия с таблицей игроков базы данных:

- получение состава команды по идентификатору;
- добавление нового игрока в команду;
- удаление игрока из команды.

Методы взаимодействия с таблицей организаторов базы данных:

- получение организатора по идентификатору;
- получение организатора по идентификатору пользователя.

Методы взаимодействия с таблицей пользователей базы данных:

- регистрация нового пользователя;
- авторизация пользователя.

3.3.2 Пользовательский интерфейс

После запуска серверной и пользовательской частей, пользовательский интерфейс станет доступен по ссылке <http://localhost:5173/>.

На рисунках 3.2, 3.3, 3.4 представлены страницы для поиска турниров, панель с функциями организатора и страница добавления турнира.

Введите название турнира Активный Поиск				
Название	Вид спорта	Дата начала	Статус	Призовой фонд
Чемпионат ст. Алдан (г...	Футбол	22.12.2025	Активный	484081.35 Р
Чемпионат к. Агата (че...	Баскетбол	24.10.2026	Активный	22428.66 Р
Чемпионат д. Павловск...	Теннис	02.02.2027	Активный	234601.48 Р
Турнир д. Середниково...	Футбол	17.02.2026	Активный	128914.45 Р
Лига к. Чокурдах (города)	Баскетбол	03.03.2026	Активный	40923.95 Р
Открытый кубок к. Мал...	Хоккей	22.02.2025	Активный	168875.12 Р
Чемпионат с. Кунгур (л...	Баскетбол	15.01.2027	Активный	351184.93 Р
Кубок с. Агата (любите...	Футбол	08.02.2026	Активный	13111.29 Р
Лига д. Мыс Шмидта (н...	Хоккей	12.03.2025	Активный	387462.33 Р

Рисунок 3.2 — Схема алгоритма триггера, обновляющего статус турнира после конца матча-финала

Панель управления

Создать турнир

Изменить информацию о турнире

Рассмотреть заявки

Добавить матч

Изменить информацию о матче

Рисунок 3.3 — Схема алгоритма триггера, обновляющего статус турнира после конца матча-финала

Турниры

Матчи

Система Управления Спортивными Турнирами

Log out

Управление

Создать новый турнир

Название турнира

Название турнира

Тип спорта

Вид спорта

Дата начала

ДД.ММ.ГГГГ

Дата окончания

ДД.ММ.ГГГГ

Призовой фонд

0

Статус

Регистрация

Сохранить Турнир

Рисунок 3.4 — Схема алгоритма триггера, обновляющего статус турнира после конца матча-финала

ЗАКЛЮЧЕНИЕ

Все задачи решены:

Цель работы достигнута:...

Имеются перспективы развития проекта. В частности, в перспективе планируется расширение, например то что после завершения турнира организаторам дается некоторое время на внесение результатов и потом турнир закрывается, но пока используется базовая версия.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. <https://nagradion.ru/>
2. <https://www.goalstream.org/>
3. <https://joinsport.io/>
4. Работа с базами данных. Лекция 5: Модели организации баз данных. // НОУ ИНТУИТ. [Электронный ресурс]. URL: <https://intuit.ru/studies/courses/3439/681/lecture/14023>
5. Типы баз данных в 2026: 13 категорий — полное руководство. // IT for Prof. [Электронный ресурс]. URL: <https://itforprof.com/infrastruktura/tipy-baz-dannyh-2026/>
6. Дейт К. Дж. Введение в системы баз данных: 8-е издание. — М.: «Вильямс», 2006. — 1328 с.
7. Кузнецов С. Д. Основы современных баз данных. — М.: Центр Информационных Технологий, 1998.
8. TypeScript Documentation [Электронный ресурс]. URL: <https://www.typescriptlang.org/docs/>
9. Express Documentation [Электронный ресурс]. URL: <https://expressjs.com/>
10. Документация языка разметки HTML [Электронный ресурс]. URL: <https://developer.mozilla.org/en-US/docs/Web/HTML>
11. Документация языка таблиц стилей CSS [Электронный ресурс]. URL: <https://developer.mozilla.org/en-US/docs/Web/CSS>
12. Vite Documentation [Электронный ресурс]. URL: <https://vite.dev/guide/>

13. PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/>
14. Redis Documentation [Электронный ресурс]. URL: <https://redis.io/docs/>
15. Visual Studio Code Documentation [Электронный ресурс]. URL: <https://code.visualstudio.com/docs>

ПРИЛОЖЕНИЕ А

<тут всякие слишком большие ништяки, которые вы хотели бы добавить
в отчет>